ИТОГОВЫЙ ПРОЕКТ

«Веб-мессенджер iMSITChat»

Срок выполнения: 1,5 месяца
Форма выполнения: парная работа (2 человека)
Формат: сквозной проект
Состав проекта: backend, frontend, база данных, локальный запуск

---

1. Общие положения

Итоговый проект выполняется обучающимися в рамках освоения дисциплин, связанных с разработкой веб-приложений, баз данных и клиент-серверных систем.

Проект выполняется **в парах (по 2 человека)**.
Оба участника пары несут **равную ответственность** за архитектуру, реализацию, запуск и защиту проекта.

Разработка и проверка проекта осуществляются **исключительно в локальной среде**.

---

2. Цель и задачи проекта

2.1. Цель проекта

Целью итогового проекта является разработка **современного веб-мессенджера**, реализованного по клиент-серверной архитектуре и обеспечивающего обмен сообщениями между пользователями.

2.2. Задачи проекта

В рамках выполнения проекта необходимо:

- реализовать регистрацию и авторизацию пользователей;
- обеспечить создание личных диалогов;
- реализовать обмен текстовыми сообщениями между пользователями;
- обеспечить автоматическое получение новых сообщений;
- реализовать хранение данных в реляционной базе данных;
- обеспечить соответствие приложения современным требованиям к безопасности и пользовательскому интерфейсу.

---

3. Технологический стек

Выбор конкретных технологий осуществляется обучающимися самостоятельно с учётом указанных ограничений. Используемые технологии должны соответствовать уровню подготовки Middle.

3.1. Языки программирования

Допускается использование следующих языков программирования:

- JavaScript / TypeScript;
- Python;
- Java;
- C#;
- PHP;
- Go;
- Kotlin.

3.2. Серверная часть (backend)

Для реализации серверной части допускается использование одного из следующих фреймворков:

- Node.js: NestJS, Express, Fastify;
- Python: FastAPI, Django, Flask;
- Java: Spring Boot;
- C#: ASP.NET Core;
- PHP: Laravel;
- Go: Gin, Fiber.

3.3. Клиентская часть (frontend)

Для реализации клиентской части допускается использование одного из следующих фреймворков:

- React;
- Vue 3;
- Angular;
- Svelte (при уверенном владении).

3.4. Пользовательский интерфейс и стилизация

Для оформления пользовательского интерфейса допускается использование:

- TailwindCSS;
- Material UI, Ant Design, Vuetify, Angular Material;
- Bootstrap 5.

3.5. База данных

В проекте должна использоваться реляционная система управления базами данных:

- PostgreSQL (рекомендуется);
- MySQL или MariaDB.

3.6. Обновление сообщений

В проекте должно быть реализовано автоматическое получение новых сообщений без ручной перезагрузки страницы с использованием одного из следующих подходов:

- WebSocket;
- Server-Sent Events (SSE);
- Long polling (с обоснованием выбранного решения).

3.7. Инфраструктура и сборка

Проект должен включать:

- Docker-файлы и docker-compose для запуска всех компонентов;
- систему непрерывной интеграции (GitHub Actions или аналог).

---

4. Функциональные требования

4.1. Пользователи

Приложение должно обеспечивать:

- регистрацию пользователей;
- авторизацию пользователей;
- хеширование паролей;
- использование ролей пользователей (user, admin).

4.2. Диалоги и сообщения

Приложение должно обеспечивать:

- создание личных диалогов формата «один на один»;
- отправку текстовых сообщений;
- хранение истории сообщений в базе данных;

- отображение даты и времени отправки сообщений.

---

5. Архитектурные требования

Проект должен включать следующие архитектурные элементы:

1. ER-диаграмму предметной области;
2. нормализованную реляционную базу данных;
3. слоистую архитектуру серверной части, включающую:
    - уровень обработки запросов (Controller, Handler);
    - уровень бизнес-логики (Service);
    - уровень доступа к данным (Repository, DAO);
4. REST API для взаимодействия клиентской и серверной частей;
5. одностраничный пользовательский интерфейс (SPA);
6. асинхронное взаимодействие между клиентской и серверной частями.

---

6. Требования к пользовательскому интерфейсу

Пользовательский интерфейс приложения должен соответствовать современным требованиям и обеспечивать удобство использования.

Интерфейс должен включать:

- навигацию в формате SPA (страницы входа, списка диалогов и чата);
- адаптивную вёрстку для мобильных и настольных устройств;
- отображение сообщений в виде диалоговых блоков;
- автоматическую прокрутку чата при появлении новых сообщений;
- обработку состояний загрузки, ошибок и пустых списков;
- валидацию пользовательских форм;
- единую дизайн-систему;
- корректную типографику и цветовую схему;
- базовые требования доступности интерфейса.

---

7. Требования к безопасности

В проекте должны быть реализованы следующие меры безопасности:

- хеширование паролей с использованием современных алгоритмов;
- JWT-аутентификация или безопасные серверные сессии;
- защита от SQL-инъекций;
- защита от XSS-атак;
- корректная настройка CORS;
- ограничение частоты запросов к API;
- проверка прав доступа пользователей к диалогам и сообщениям.

8. Тестирование и документация

Проект должен включать:

✓	не менее пяти модульных (юнит) тестов;
✓	не менее двух интеграционных тестов;
✓	файл README с описанием структуры проекта и инструкцией по запуску;
✓	описание используемого технологического стека.

9. Условия выполнения и проверки проекта

Проект выполняется и проверяется **исключительно в локальной среде**.

Студенты не обязаны:

➢	арендовать серверы или хостинг;
➢	публиковать проект в сети Интернет;
➢	оплачивать домены, облачные базы данных или платные API.

Проект должен корректно запускаться:

➢	на компьютере преподавателя;
➢	без постоянного доступа к сети Интернет (за исключением установки зависимостей);
➢	без регистрации в сторонних сервисах.

Время запуска проекта не должно превышать 5–10 минут.

10. Использование внешних API

При использовании внешних API допускается:

- ✓ использование бесплатных API;
- ✓ использование mock-сервисов или локальных заглушек.

Приложение должно корректно работать при отсутствии доступа к внешнему API.

Использование платных API запрещено.

11. Критерии оценки

11.1. Требования для получения зачёта (оценка «3»)

Для получения зачёта проект должен соответствовать всем следующим требованиям:

✓	реализована регистрация и авторизация пользователей;
✓	обеспечено хеширование паролей;
✓	реализовано REST API;
✓	обеспечена защита от доступа к чужим чатам;
✓	реализованы личные диалоги;
✓	обеспечена отправка и получение сообщений;
✓	реализовано автоматическое обновление сообщений;
✓	реализован современный пользовательский интерфейс;
✓	проект запускается с использованием Docker и docker-compose;
✓	предоставлена инструкция по запуску проекта;
✓	тесты запускаются корректно.

При невыполнении хотя бы одного требования проект считается невыполненным.

11.2. Требования для оценок «4–5»

Для получения повышенной оценки дополнительно учитываются:

✓	качество архитектурных решений;
✓	чистота и структурированность кода;
✓	наличие логирования и единого формата ошибок;
✓	расширенные элементы пользовательского интерфейса;
✓	дополнительные функциональные возможности чата;
✓	расширенное тестирование;
✓	качество документации;
✓	стабильность работы системы.

---

12. Порядок защиты проекта

Защита проекта проводится в форме демонстрации работающего приложения и ответов на вопросы преподавателя.

Каждый участник пары должен продемонстрировать понимание архитектуры проекта и реализованного функционала.